

Solutions - TD 03 (Procedures & Functions)

Exercise 01: Parameter passing (By value / By address / Mixed)

For each algorithm we show the pseudocode and the outputs (values inside the procedure and in the main program).

1) Test_ByValue (pass by value)

Pseudocode:

Program Test_ByValue

Var a, b: Integer

Procedure Modify(x: Integer; y: Integer)

x := x + 10

y := y + 20

Write("Inside procedure: x = ", x, ", y = ", y)

EndProcedure

Begin

a := 5

b := 15

Call Modify(a, b)

a := a + 1

b := b + 1

Write("In main program: a = ", a, ", b = ", b)

End

Output explanation: Inside procedure, x = 15, y = 35. In main program, original a and b unchanged by procedure (pass-by-value) then increased by 1: a = 6, b = 16.

2) Test_ByAddress (pass by address/reference)

Pseudocode:

Program Test_ByAddress

Var a, b: Integer

Procedure Modify(Var x: Integer; Var y: Integer)

x := x + 10

y := y + 20

Solutions - TD 03 (Procedures & Functions)

```
Write("Inside procedure: x = ", x, ", y = ", y)
EndProcedure
```

```
Begin
a := 5
b := 15
Call Modify(a, b)
a := a + 1
b := b + 1
Write("In main program: a = ", a, ", b = ", b)
End
```

Output explanation: Inside procedure, x = 15, y = 35. Because parameters are by reference, a and b are modified. After the procedure a=15, b=35. Then main adds 1: a = 16, b = 36.

3) Test_Mixed (first by value, second by reference)

Pseudocode:

```
Program Test_Mixed
Var a, b: Integer

Procedure Modify(x: Integer; Var y: Integer)
x := x + 5
y := y * 2
Write("Inside procedure: x = ", x, ", y = ", y)
EndProcedure

Begin
a := 4
b := 6
Call Modify(a, b)
a := a + 10
b := b + 3
Write("In main program: a = ", a, ", b = ", b)
End
```

Solutions - TD 03 (Procedures & Functions)

Output explanation: Inside procedure: $x = 9$ (local only), $y = 12$ (b is doubled by reference). After return, a remains 4 then main adds 10 $\rightarrow a = 14$. b was changed to 12 in procedure, then main adds 3 $\rightarrow b = 15$.

4) Test_all_ByAddress (both by reference): same as Test_ByAddress.

Exercise 02: Swap procedure

Goal: Create a procedure Swap(a,b) that exchanges values. Compare pass-by-value and pass-by-reference.

Pass-by-value version (will NOT change main variables):

```
Procedure Swap(x: Integer; y: Integer)
```

```
  temp := x
```

```
  x := y
```

```
  y := temp
```

```
EndProcedure
```

```
Begin
```

```
  a := 3; b := 7
```

```
  Call Swap(a, b) // values in main remain a=3, b=7
```

```
End
```

Explanation: Since x and y are copies, swapping them does not affect a and b outside the procedure.

Pass-by-reference version (will change main variables):

```
Procedure Swap(Var x: Integer; Var y: Integer)
```

```
  temp := x
```

```
  x := y
```

```
  y := temp
```

```
EndProcedure
```

```
Begin
```

```
  a := 3; b := 7
```

```
  Call Swap(a, b) // after call a=7, b=3
```

```
End
```

Solutions - TD 03 (Procedures & Functions)

Explanation: Using reference parameters swaps the original variables in the main program.

Exercise 03: Increment function vs. procedure

1) Function Increment(n): returns $n + 1$ (does not change caller variable unless assigned).

```
Function Increment(n: Integer) : Integer
```

```
    Return n + 1
```

```
EndFunction
```

```
Begin
```

```
    a := 5
```

```
    b := Increment(a) // b = 6, a remains 5
```

```
End
```

2) Procedure IncrementRef(Var n): adds 1 directly to the variable.

```
Procedure IncrementRef(Var n: Integer)
```

```
    n := n + 1
```

```
EndProcedure
```

```
Begin
```

```
    a := 5
```

```
    Call IncrementRef(a) // a becomes 6 directly
```

```
End
```

Comparison: Function returns a value and leaves original unchanged unless you assign it back. Procedure with reference modifies the original variable.

Solutions - TD 03 (Procedures & Functions)

Exercise 04: Compute sum and average using pass by reference

Pseudocode:

```
Procedure Compute(Var a: Integer; Var b: Integer; Var sum: Integer; Var avg: Real)
  Read(a)
  Read(b)
  sum := a + b
  avg := sum / 2.0
EndProcedure
```

```
Begin
  Var a, b: Integer; sum: Integer; avg: Real
  Call Compute(a, b, sum, avg)
  Write("Sum = ", sum)
  Write("Average = ", avg)
End
```

Explanation: Read two integers inside procedure, compute sum and average, results are written back through reference parameters.

Exercise 05: Square function

1) Function Square(x) returns $x * x$.

```
Function Square(x: Integer) : Integer
  Return x * x
EndFunction
```

```
Begin
  Read(n)
  s := Square(n)
  Write("Square = ", s)
End
```

Solutions - TD 03 (Procedures & Functions)

Exercise 06: Max function

Pseudocode:

```
Function Max(a: Integer; b: Integer) : Integer
  If a >= b Then
    Return a
  Else
    Return b
  EndIf
EndFunction
```

```
Begin
  Read(x); Read(y)
  m := Max(x, y)
  Write("The larger is: ", m)
End
```

Exercise 07: IsEven function

```
Function IsEven(n: Integer) : Boolean
  Return (n mod 2 = 0)
EndFunction
```

```
Begin
  Read(n)
  If IsEven(n) Then
    Write(n, " is even")
  Else
    Write(n, " is odd")
  EndIf
End
```

Solutions - TD 03 (Procedures & Functions)

Exercise 08: Factorial function

Iterative and recursive versions (pseudocode):

Iterative:

Function Factorial(n: Integer) : Integer

 If $n < 0$ Then Return -1 // error; factorial not defined for negative in this simple version

 result := 1

 For i := 2 To n Do

 result := result * i

 EndFor

 Return result

EndFunction

Recursive:

Function Factorial(n: Integer) : Integer

 If $n = 0$ Or $n = 1$ Then

 Return 1

 Else

 Return $n * \text{Factorial}(n - 1)$

 EndIf

EndFunction

Test examples: Factorial(0)=1, Factorial(5)=120.

Exercise 09: Temperature conversion functions

Pseudocode:

Function CelsiusToFahrenheit(c: Real) : Real

 Return $(9.0 / 5.0) * c + 32.0$

EndFunction

Function FahrenheitToCelsius(f: Real) : Real

 Return $(5.0 / 9.0) * (f - 32.0)$

EndFunction

Example usage: Read a Celsius value, convert and display both.

Solutions - TD 03 (Procedures & Functions)

Exercise 10: SumN function

Pseudocode (iterative):

```
Function SumN(n: Integer) : Integer
    sum := 0
    For i := 1 To n Do
        sum := sum + i
    EndFor
    Return sum
EndFunction
```

// Or closed form: return $n * (n + 1) / 2$

Example: SumN(5)=15.

Exercise 11: Square and Cube functions

Pseudocode:

```
Function Square(x: Integer) : Integer
    Return x * x
EndFunction
```

```
Function Cube(x: Integer) : Integer
    Return x * x * x
EndFunction
```

```
Begin
    Read(n)
    Write("Square = ", Square(n))
    Write("Cube = ", Cube(n))
End
```

Solutions - TD 03 (Procedures & Functions)

Exercise 12: Sum and Average using functions

Pseudocode:

```
Function Sum(x: Integer; y: Integer) : Integer
  Return x + y
EndFunction
```

```
Function Average(x: Integer; y: Integer) : Real
  s := Sum(x, y)
  Return s / 2.0
EndFunction
```

```
Begin
  Read(a); Read(b)
  s := Sum(a, b)
  av := Average(a, b)
  Write("Sum = ", s)
  Write("Average = ", av)
End
```

Exercise 13: Student grades: average and pass/fail

Pseudocode:

```
Function ComputeAverage(g1: Real; g2: Real; g3: Real) : Real
  Return (g1 + g2 + g3) / 3.0
EndFunction
```

```
Function IsPassed(g1: Real; g2: Real; g3: Real) : Boolean
  avg := ComputeAverage(g1, g2, g3)
  Return (avg >= 10.0)
EndFunction
```

```
Begin
  Read(g1); Read(g2); Read(g3)
  avg := ComputeAverage(g1, g2, g3)
  If IsPassed(g1, g2, g3) Then
```

Solutions - TD 03 (Procedures & Functions)

```
    Write("Passed. Average = ", avg)
Else
    Write("Failed. Average = ", avg)
EndIf
End
```